The hidden technical debt of agentic engineering

Agents are easy to build but hard to run. At Port, we mapped seven blocks of hidden infrastructure debt with AI agents in enterprise systems.

Zohar Einy

Anyone today can build an agent locally with minimal effort. With some LLM calls, a prompt, and a few tool definitions, that agent will be doing real work for them within minutes. But what happens when that agent needs to get into production and be used by the entire engineering department, with real data and real consequences?

In 2015, Google published "Hidden Technical Debt in Machine Learning Systems." That paper turned on the light for machine learning engineers and named all the problems they were experiencing. The diagram they shared also became iconic: a tiny box labeled "ML Code" surrounded by massive infrastructure blocks.

We're seeing the same pattern for agents. Agents are a small part of the picture, and we want to name all the infrastructure around them.

Agentic engineering systems are especially good at piling up technical debt. They carry all the maintenance problems of traditional software, plus an additional set of agent-specific issues. New agents are being created daily by nearly every employee. Soon you'll have far more agents than employees.

We define an agent as any process with dynamic decision-making capabilities that can autonomously determine tool usage and execution paths through reasoning and reflection. Decision-making, reasoning, and reflection require all the supporting infrastructure.

Building an agent is easy. But in production, the agent code is the smallest part of the system. Everything around it is where the actual complexity lives.

TRENDING STORIES

Over the past few months, through conversations with engineering leaders and our own experience, we've mapped out seven infrastructure blocks that surround the agent. Each one is a category of work that no one plans for when building their demo.

Some of these blocks will look familiar if you've done traditional engineering: observability, integrations, and governance. Others are unique to agents, such as human-in-the-loop, evals for non-deterministic systems, and the agent registry.

Let’s walk through each one.

1. Integrations

Agents need to reach your actual systems: CI/CD, cloud providers, incident tools, observability platforms, code repos, secret managers, and more.

Without centralized integrations, every team wires up its own connections to agents.

Picture an engineering org with 200 engineers on 30 teams, each with multiple agents. Each engineer generates their own GitLab PAT for coding agents, Snowflake credentials for data agents, Kubernetes service accounts for deployment agents, and monitoring tokens for incident agents.

That’s hundreds of integration points, each one configured individually, debugged individually, and expiring on its own schedule.

When every developer wires their own credentials, each agent sees different data depending on whose token it uses. One developer's GitLab PAT has access to all repos. Others are scoped to their team. Same agent type, but each has a completely different view of the org.

Or, what happens when GitLab ships a breaking change to their API? Every team that wired its own connection independently debugs the same issue (or submits a ticket to the platform team). Three teams figure it out on Monday. Two more by Wednesday. One team doesn't notice for a week because their agent only runs during incidents.

What also matters is what comes through those integrations. When three teams connect to the same data source through different paths, their agents can get different answers to the same question. If one team's integration pulls a 30-day history of deployments, and another team's integration shows everything from the last 3 years, their outputs will differ.

Right now, MCP is how most teams connect agents to their tools. But let's not confuse MCPs with integrations. MCP gives agents a standard way to call a tool. It doesn't manage the credentials for that call, the scope of the data that comes back, or what happens when the API on the other end changes.

Hidden tech debt in integrations looks like:

- An integration auth token expires on a Friday night, and an incident agent silently stops working. Nobody notices until Monday.
- Five teams each maintain their own GitLab connection with different permissions and scopes, unaware that the others

exist.
- When an integration updates its API, and every team debugs its connection separately.

2. Context lake

Agents are only as good as the context they can reference and use. They need two kinds of context.

Runtime context: How do you deliver accurate context to agents during their execution?

The runtime context is the live data that agents need for a specific execution, such as information about services, who owns them, and what was deployed recently. It's the same kind of data humans use when coding or resolving incidents, but more accessible to agents.

Think of a coding agent picking up a ticket to add a retry mechanism to a service. It needs to know: what language and framework the service uses, how retries are handled in other services in this org, who owns the downstream service it calls, and whether there's been a recent config change to the timeout settings.

Some teams manage their runtime context in markdown files: agents.md, .cursorrules, skills files.

Markdown files are fine for static instructions, such as how to format commits or which linter to use. But the runtime context changes constantly. Service ownership shifts. Dependencies get added. Config values update. Deployments happen every hour. An agent running on a .md file that says "checkout-service is owned by Team Payments" doesn't know that ownership transferred to Team

Commerce last Tuesday. The file was accurate when it was written. By the time the agent reads it, it might not be.

Decision traces: How do you help agents learn from their own past work or the work of other agents?

Decision traces are a history of what has been done before (by humans or agents), why it was decided, and what happened afterwards. Without that history to reference, every agent run starts from zero. Think of an agent opening a PR to fix a flaky test. It doesn't know that a different agent tried the same fix last week, the PR was rejected because it broke a downstream contract, and the team decided to deprecate the test entirely. So it reopens the same PR. Without decision traces, agents repeat mistakes that humans (or agents) have already resolved.

An agent that resolved 50 incidents has seen patterns a new agent hasn't, like which fixes worked, which ones caused regressions, and which services are fragile after deploys. Without traces, that knowledge disappears after every run. When multiple agents operate on the same systems, they can't see each other's history.

LLM providers are starting to address this with `memory.md` files that can be shared across teams. But the debt still shows up when you have dozens of agents operating. You need to find a way to reliably serve that memory (or just the right parts of it) to specific agents.

Hidden tech debt in context lake looks like:

- Stale, fragmented context that no one owns
- Agents running on agents.md when company standards live in a wiki

- Agents not learning how and why other agents solved a problem, or about the mistakes they made

3. Agent registry

Gaining visibility into what agents exist

The org chart is changing. Instead of just people, you now have 5–10 times that number of agents. They're being created daily by all your human employees. They're running without guardrails, they have access to critical infrastructure, and they're making decisions. They're also spread across tools like Claude Code, Cursor, n8n, zapier, Notion, AWS, GCP, and more.

The typical pattern goes like this: An engineer builds a triage agent, and their team starts using it to help with incidents. Another team builds its own version because it didn't know the first one existed. A third team builds something similar but wired to different tools with different permissions.

In a company with 20 or 30 engineering teams, you'll quickly reach agents with overlapping responsibilities, conflicting behaviors, and invisible dependencies. Before agents can be shared between teams, you need to know they exist.

Delivering instructions to agents

Once you have visibility into your agents, they need the equivalent of an employee handbook: standards, skills, and instructions on how they're expected to operate.

Today, engineers create skill files for their agents independently. The issue is that when they are scattered across repos without a

centralized view, teams end up creating duplicate or inaccurate skills. They often contradict the platform-distributed context. The platform team has far more insight into what to write in skill files than individual teams do.

So how do you get consistent but personalized coding rules, commands, skills, and hooks to the right agents?

This information may even require multiple levels:

- company-wide standards that apply everywhere (secure coding, commit conventions)
- repo-specific instructions ("in this repo, events are created this way")
- Or team-level rules for subsets of engineers

You'll need to find a way to reliably deliver this information to thousands of agents, ensuring the right instructions reach the right agents.

Agent creation

Let's say you've gotten visibility on most existing agents and delivered instructions to them. The next question is: How can you control the process of creating new agents without slowing your team down?

That responsibility now falls on platform teams.

Like services in a software catalog, agents should have standardized properties and should be connected to other entities in your company, like other agents, teams, services, deployments, etc.

Without a template, you get the same sprawl problem you just solved. An engineer spins up an agent with no owner, no lifecycle state, and no connection to the service it operates on. It works for them. Nobody else knows it exists. Six months later, someone finds it running in production with expired tokens and no way to contact whoever built it.

> "An engineer spins up an agent with no owner, no lifecycle state, and no connection to the service it operates on. It works for them. Nobody else knows it exists."

Agent creation should follow a standardized template. This doesn't mean that human employees should be slowed down when they create agents. The opposite is true. By having a standardized process for creating agents, you will help them reach high-quality work faster than if they were created individually.

Agent creation should be allowed from the engineers' workstations. If they work in Cursor and need to spin up an agent, they should be able to do it from there, not elsewhere. Which means your job, as platform engineers, is to make sure agents created anywhere follow your creation process.

A template doesn't restrict what the agent does. It makes sure every agent is born with the basics: an owner, a description, the tools it uses, the services it touches, and a lifecycle state. That's what makes it governable from day one, rather than something you have to chase down later.

Hidden tech debt in the agent registry looks like:

- Invisible agents
- Teams are creating duplicate agents

- Outdated context
- When the CISO asks for an agent audit, but there's no list to start from
- No clear way of promoting an agent to production
- No versioning, rollbacks, or staging environments

4. Measurement

How do you know if your agents are working? Well, it depends on who's asking.

An SRE wants to know what the agent did.

An ML engineer or product manager wants to know if it's getting better or worse.

A VP of engineering wants to know if it's worth the money.

An end user wants to know that the agent is learning based on their feedback.

So while each person wants to measure something in agents, they all want different types of measurement.

1. How do you know what your agents are doing?

This is **observability.**

Events, traces, and logs show what actions the agent took, what data it had access to, and if it is still running correctly. Engineering teams understand observability from traditional systems, but with agents, the surface area is wider.

Say you have an agent that autonomously solves Jira tickets. When

it receives a Jira ticket, it reads the service catalog to find the relevant repo, pulls recent commits from a GitHub integration, generates a fix with a coding agent, opens a PR back in GitHub, and requests review from the service owner that it found in the software catalog. If the fix broke something, which step went wrong? Did it pull the wrong repo? Misread the ownership? Generate bad code?

If you can't trace individual links in that entire chain, good luck debugging it.

2. How do you know if your agents are getting better or worse when prompts, skills, tools, and models change?

This is **evals**

In standard software engineering, you can write a unit test that expects an exact string. In agentic engineering, when every response is different, you need a different approach.

Evals answer a simple question: after you change something (like a prompt or model), is the agent still good?

Without a way to track this, changes go out untested, and output quality degrades silently. Like swapping Sonnet for Opus, and your PR review agent starts approving things it used to flag.

3. How do you know if your agents are actually working for your business?

This is **business impact**

Every earnings call this year will include the question: "What is AI doing for your business?".

Most engineering leaders can't answer it yet, and ultimately, they are the ones responsible for measuring agent cost and ROI.

Tracking spend is the easier part of this equation. You can track token usage, API calls, and compute costs per agent, per team.

ROI is harder to measure. How many tickets did agents resolve? How much engineering time did they save? Did it actually reduce MTTR or just move work around? These numbers are harder to collect and harder to trust. If you can only show the cost side without showing a clear ROI, that's a bad conversation to be having.

4. How do you give agents feedback on their work?

These are **feedback loops**

When an agent generates a PR, resolves a ticket, or writes an RCA, did the human who reviewed it accept the output or correct it? This is generally managed with a thumbs-up or thumbs-down, but sometimes it's the human's response itself that counts as feedback (something like "No, try again, but do change X instead").

They're critical for improving the agent and more important than evals. Agents in the demo phase either don't collect these signals or don't act on them.

Hidden tech debt in measurement looks like:

- Not knowing if agent performance is improving or declining over time, and compared to what
- Not being able to measure what happens when a prompt or model changes
- Leadership is asking for ROI, but you don't have a clear answer

- Failure to collect feedback from humans using agents

5. Human-in-the-loop

There's a spectrum between fully manual and fully autonomous engineering. On one end, a human does everything. On the other end, an agent acts without asking. Most useful agents live somewhere in the middle, and where exactly they sit depends on the action, the environment, and the risk.

Human-in-the-loop is one of the mechanisms that lets you move agents closer to autonomous safely. It lets you define checkpoints: this action needs approval, this one doesn't, and this one depends on the environment. The agent runs, but a human confirms the high-stakes decisions before they execute.

For example, a deployment agent might run freely in staging but require approval in production. It can be fully autonomous during business hours, but we need a human in the loop at 3 a.m. The rules are conditional and vary by agent, action, environment, and team.

When you have one agent in a demo, you can hard-code the approval checkpoints as an if statement that pings Slack before a deploy. With 100 agents across 20 teams, hard-coded approval logic doesn't scale. Every team implements its own version. One team's agent rolls back production without asking. Another team's agent requires three approvals to do the same thing. Nobody defined this centrally, so nobody knows which agents can act on their own and which can't.

And then there's the orchestration of the approvals themselves. Who gets notified? Through what channel? What's the timeout if

nobody responds? What happens if the approver is on vacation? If approval comes through Slack for one agent, via email for another, and via a custom UI for a third, you now have three approval systems to maintain. The logic around approvals becomes major tech debt, separate from the agents themselves.

If we zoom out, human-in-the-loop is also about visibility into what's happening inside all the agents working for us. As engineers move into a more managerial role (of agents), they will need a control plane to see what's in motion, initiate agent work, identify which agents need attention, and take action if needed.

This matters because human-in-the-loop is how you apply change at scale (and companies today either change or die). To succeed in their new job, engineers need to see the agents working, just as they used to see their code working. A team that can watch an agent handle its first ten deployments (see every step, review every decision, and intervene when needed) will trust that agent. A team that can't, simply won't.

Hidden tech debt in human-in-the-loop looks like:

- Hard-coded approval code that can't be changed from one place
- Some agents run without approvals, others have too many
- Multiple approval systems by email, Slack, and custom UI that don't work with each other
- No shared workspace where teams can see what their agents are doing and intervene if necessary

6. Governance

When a human engineer needs access to a production database,

there's a process. They submit a request, someone approves it, and the access is scoped and logged. It might take an hour or a day, but there's a record of who has access to what and an audit log of who did what.

When an engineer creates an agent locally, it usually runs with whatever credentials its creator wired in: their API tokens, their service account, their cloud permissions. Most likely, nobody reviewed the scope.

Governance rules for agents need to be specific:

- "Roll back a service, but only if there's a high-severity incident open."
- "Deploy to production requires manual approval, always, no matter what agent triggered it."
- "An RCA report that pulls data from external systems should only be visible to the owners of that service."

These are the kinds of rules that platform teams need to define in a single centralized place and apply across all agents.

The other side of governance is enforcement. Say you discover a vulnerability in one of your internal APIs and need to block all agents from calling it immediately. Can you? One security engineer should be able to disable a tool in one place and have it automatically disabled across all agents. At most companies, that capability doesn't exist yet.

You can't always get access permissions airtight from the start. Things can go wrong, and if they do, you need to know what happened: which agent took the action, what data it accessed, what credentials it used, and who triggered it. Most agent setups

don’t produce that audit trail (certainly not if they’re created locally). An agent running locally inherits its creator’s credentials, so every action appears to be the engineer’s personal work. If three agents share a service account, you can’t tell which one made the call. If an agent chains through two other agents before modifying a production config, the audit log might show only the final write, not the reasoning, context, or the chain of decisions that led to it.

Another aspect of governance is cost governance. Agents tend to keep working despite the costs they are racking up. When an engineer creates an agent locally, they probably won’t think about setting a cost limit.

An agent stuck in a retry loop or reasoning in circles will keep burning tokens for hours until someone manually kills it or until the monthly invoice shows the damage. Most teams can tell you their total LLM spend, but almost none can break it down by agent, by team, or by use case. Teams should also be able to easily see where their costs stand. Because when leadership asks what agents cost to run, engineering will need an answer.

Hidden tech debt in governance looks like:

- An agent that shouldn’t run in production, accessing production data
- An RCA agent is publishing sensitive service data to a shared channel
- An agent publishing PII in a public forum
- No way to disable a tool across all agents from one place
- No audit trail of what an agent did or why

7. Orchestration

Most agentic workflows aren’t purely agents. They’re a mix of agents, tools, and people. The debt isn’t in the individual steps. It’s in what happens between them: routing, failure handling, and ownership.

What breaks between the nodes

Take an incident response workflow like the one above. An alert fires. A triage agent picks it up, investigates, and determines the root cause is a deployment issue. It hands off to a deployment agent that rolls back the change. A verification agent checks that the fix worked.

Now imagine the triage agent got it wrong. The real cause was a database timeout, not a bad deploy. The deployment gets rolled back unnecessarily. The database issue persists. The alert keeps firing. Eventually, a human steps in and starts debugging from scratch, but now they're also cleaning up the rollback that shouldn't have happened. The failure wasn't silent. The workflow confidently took a wrong action, and nobody could tell where the bad decision was made.

That's what orchestration debt looks like in practice. Not necessarily workflows that stop, but workflows that do the wrong thing and are hard to trace.

And every new issue type you add later, like security incidents, config drift, or dependency failures, makes routing harder to test and explain. None of those changes is hard on its own. But when nobody owns the decision of how they connect, they get wired together differently every time.

What's different from traditional workflow orchestration

Workflow orchestration isn't new. Teams have been chaining steps together for years in CI/CD pipelines, Airflow, and Step Functions.

So why is agent orchestration a different problem?

Traditional workflows are deterministic. Step A produces a known output, step B consumes it. You can test every path because you know every path. Agent workflows introduce non-determinism into chains that previously had none. When you replace a runbook with an agent that reasons about the problem, every downstream step becomes unpredictable. You can't test every path because you don't know every path.

There's also no contract between agents. Services have APIs with schemas and versioned endpoints. Two agents handing off to each other have prompts and natural language. The "interface" is fuzzy. A model update or prompt change in one agent can shift its output just enough to break the next agent in the chain.

A deployment pipeline, for example, should be completely deterministic. The trigger fires, the system knows the severity, deploys to staging, a human approves, it goes to production, and a verification agent checks health. The steps are predefined, the order is fixed, and the risk is high enough that you want it that way.

An incident response workflow is inherently non-deterministic. The root cause could be a bad deploy, a database issue, a misconfiguration, or something nobody's seen before. The agent has to investigate and decide what happens next.

Most engineering teams need both kinds of workflows. The debt is that there are no shared rules about when to use which. One team sends every triage result straight to a coding agent. Another requires human review before any automated fix. Both work in isolation until an incident crosses a team boundary and the two approaches collide.

Who owns the workflow?

Even if every individual agent has an owner, that's not enough. The orchestration itself needs an owner.

When an agent modifies a service, and something breaks, who's responsible? Is it the agent's owner or the service owner? When a workflow spans three teams, which team owns the outcome? When a step fails mid-chain, is it the failing agent's responsibility to retry, or the workflow's responsibility to reroute?

These aren't theoretical questions. They're the questions that come up at 2 a.m. when an agent-driven workflow takes a wrong turn, and three teams are in a war room trying to figure out whose agent did what. Individual step failure is easy to debug. Figuring out which decision, three handoffs ago, sent the workflow down the wrong path — that requires tracing that most orgs haven't built.

> "...when an agent-driven workflow takes a wrong turn, and three teams are in a war room trying to figure out whose agent did what."

Hidden tech debt in orchestration looks like:

- An agent fails mid-workflow, and nobody finds out until the downstream effect surfaces
- No way to trace a decision across agents back to the original trigger
- A workflow spans three teams, but no team owns the outcome
- A model or prompt change in one agent silently breaks the next agent in the chain

When the debt hits

This hidden tech debt gets painful at specific trigger points.

At the exploration stage, there's no debt. One engineer, one agent, it works. When a team starts using agents for real work, though, integrations and context are the first things that break. An agent accesses customer data that it shouldn't see because nobody scoped the credentials. An agent guesses a service owner because it didn't have that context.

When multiple teams run agents independently, the debt piles up faster. Agent registry, measurement, and human-in-the-loop all surface at once. At this stage, about 50% of a team's capacity would go to building the surrounding infrastructure.

At production scale (agents embedded across most of your engineering org), governance and orchestration become the priority. Some companies see it coming. One architect told me, "We know from experience that chaos will exist. We will try to avoid it from day one." Others learn the hard way: a VP of platform engineering saw teams independently building the same agents

and had to retrofit governance once the sprawl was already in place. Both will end up building the same infrastructure. One will pay for it twice.

This happened before with microservices

This moment feels similar to the evolution of microservices. Each team chose its own technology, did its own infrastructure, and eventually someone had to come in and create standards. There is another platform engineering moment happening now with agents.

Platform engineering used to be a velocity initiative: self-service, reducing the ticket load, and scaffolding new services. With agents, it's still about velocity, but the platform team is playing catch-up. Engineers aren't waiting. They're able to spin up new agents in Cursor or Claude Code whenever they need one. The platform team's first job is to identify which agents already exist and get them under control. Only then can they do what they've always done: make it faster, safer, and easier for everyone else to create and use them.

One DevEx team described their new role to me: "We won't be the ones designing and creating the agents. What we're responsible for is making sure devs know how to use them, can interact appropriately, and helping more teams create their own agents."

What to do about it

Start with visibility. Audit your GitHub org for AI-related workflows and actions. Check how many active API tokens your teams have on Claude, OpenAI, or Bedrock. Look at your workflow tools for anything with an AI node. The goal isn't a perfect inventory. It's the

first count.

The harder question is agreeing on what counts as an agent. Is a GitHub Actions automation an agent? Is it a Claude Code scheduled task? Is it an n8n workflow with an AI node? You'll need a working definition before you can catalog them. It doesn't have to be perfect, but your org needs to agree on one.

There's also the question of centralized vs democratized? Should the platform team build everything, and devs consume it? Or should the platform team provide guardrails while teams build their own? Both models exist. It probably depends on how much control your culture will tolerate.

You can build this infrastructure now, or you can build it after an agent leaks customer data, burns $300 in tokens overnight, or silently rolls back a production service nobody asked it to touch. You'll build it either way. The only question is whether you build it before the pain or after.

TNS

Zohar Einy is the CEO of Port, the agentic engineering platform that is helping customers like GitHub, Visa, and PwC move from manual to autonomous engineering. Zohar began his career in the Israel Defense Forces' 8200 unit as an engineer,...

Read more from Zohar Einy